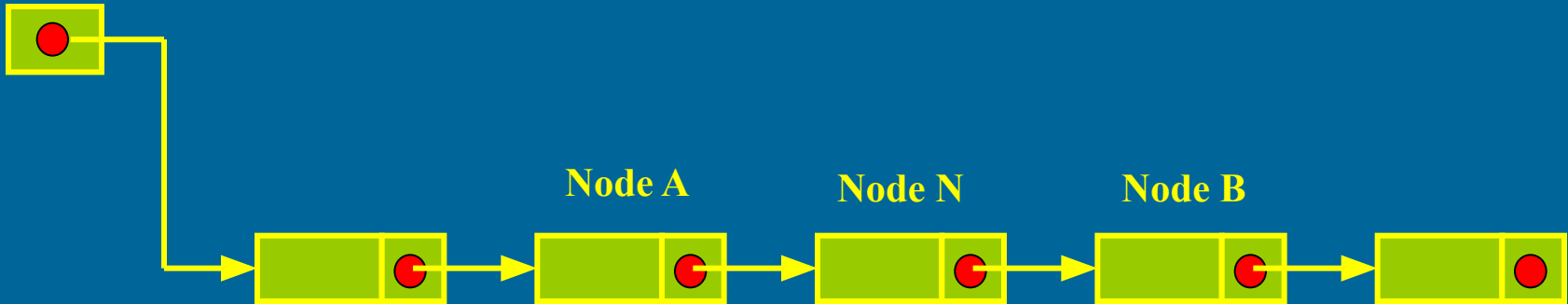


LECTURE - 4

Deletion from a Linked List

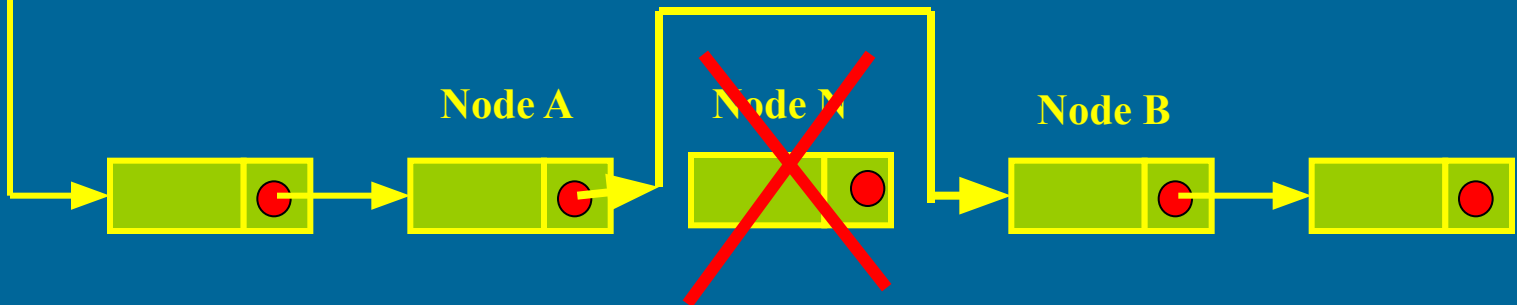
start



Before Deletion



start



After Deletion

Observe that three pointer fields are changed as follows:

1. The nextpointer field of node A now points to the new node B, where Node N previously pointed
2. The nextpointer field of N now points to the original first node in the free pool, where AVAIL previously pointed.
3. AVAIL now points to the deleted node N

There are two special cases:

1. If the deleted node N is the first node in the list, then START will point to node B
2. If the deleted node N is the last node in the list, then node A will contain the Null pointer.

Algorithm-6

DEL (Info, Link, Start, Avail, Loc, LocP)

Deleting a given node following a given node

1. if $LOC P := NULL$, then:
 set $START := LINK[START]$ [deletes first node]
else:
 set $LINK[LOC P] := LINK[LOC]$ [deletes node N]
[end of if structure]
2. [Return deleted node to the Avail List]
 set $LINK[LOC] := AVAIL$ and $AVAIL := LOC$
3. Exit.

Procedure-1

FINDB (Info, Link, Start, Item, Loc, LocP)

Deleting the node with a given item of information

1. [list empty] if START = NULL, then:
 set LOC := NULL and LOCP := NULL and return
[end of if structure]
2. [Item in first node] if INFO[START] = ITEM, then:
 set LOC := START and LOCP := NULL and return
[end of if structure]
3. Set SAVE := START and PTR := LINK[START] [initialize pointers]
4. Repeat Steps 5 and 6 while PTR ≠ NULL
5. if INFO[PTR] = ITEM, then:
 set LOC := PTR and LOCP := SAVE, and return
 [end of if structure]
6. Set SAVE := PTR and PTR := LINK[PTR] [update pointers]
[end of loop]
7. Set LOC := NULL [search unsuccessful]
8. Return

Algorithm-7

DELETE (Info, Link, Start, Avail, Item)

1. [use procedure 1 to find the location of N and its preceeding node]
Call FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
2. If LOC = NULL , then: write “ item not in list”, and exit
3. If LOCP=NULL, then:
 set START := LINK[START] [deletes first node]
Else:
 set LINK[LOCP] := LINK[LOC]
[end of if structure]
4. [return deleted node to AVAIL list]
set LINK[LOC] := AVAIL and AVAIL := LOC
5. Exit.

PRACTICE

Example 5.17

(a, b)

ON PAGE 139

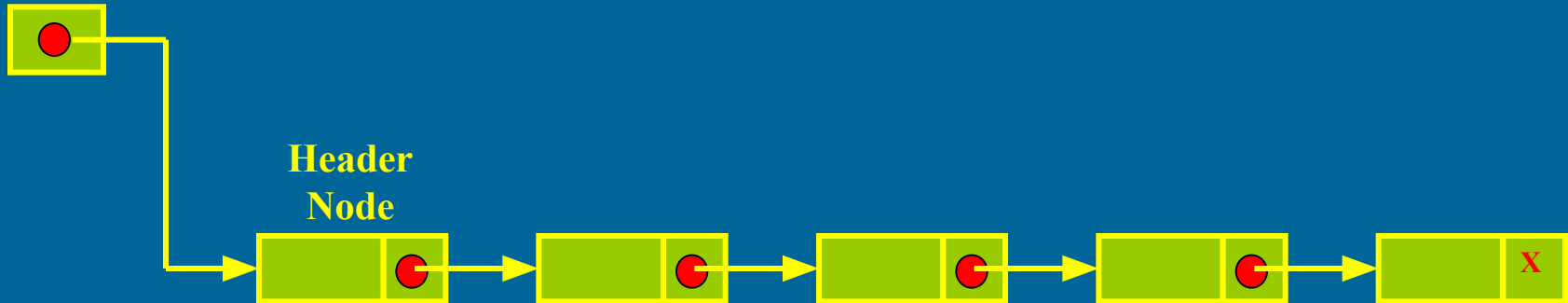
HEADER LINKED LIST

A header linked list is a linked list which always contains a special node, called the header node, at the beginning of the list.

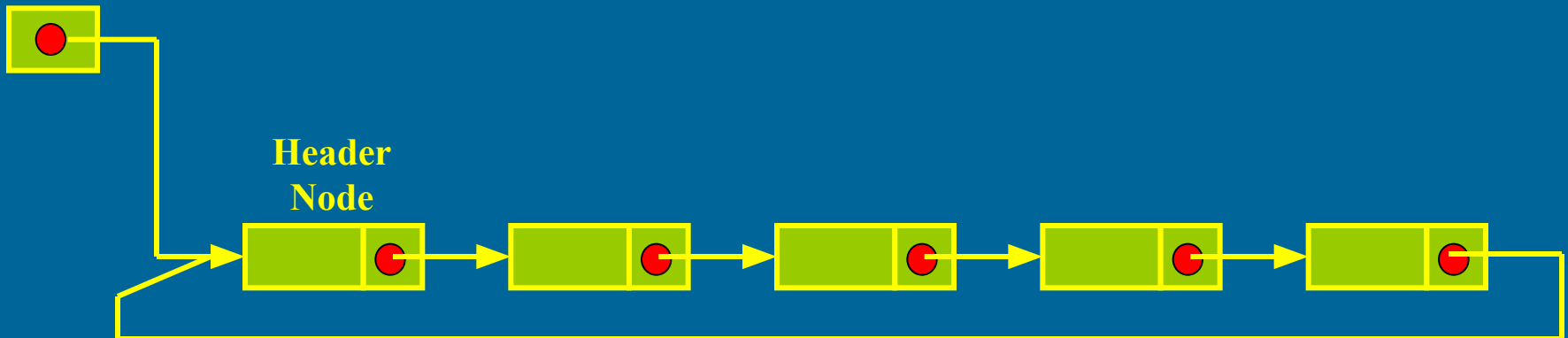
There are two kinds of widely used header list.

- A Grounded Header
- A Circular Header

A **Grounded Header List** is header list where the last node contains the null pointer.



A **Circular Header List** is a header list where the last node points back to the header node.



Algorithm-8

Traversing a Circular Header List

1. Set $PTR := LINK[START]$ [initializes pointer PTR]
2. Repeat steps 3 and 4 while $PTR \neq START$
3. Apply process to $INFO[PTR]$
4. Set $PTR := LINK[PTR]$
- [End of loop]
5. Exit

Algorithm-9

SRCHHL (Info, Link, Start, Item, Loc)

Searching in a Circular Header List

1. Set PTR := LINK[START] [initializes pointer PTR]
2. Repeat while INFO[PTR] $\neq$ ITEM and PTR $\neq$ START
 Set PTR := LINK[PTR] [PTR now points to next node]
 [end of loop]
3. If INFO[PTR] = ITEM, then:
 set LOC := PTR
else:
 set LOC := NULL
 [end of if structure]
4. Exit

PRACTICE

Procedure 5.13

ON PAGE 142

PRACTICE

Algorithm 5.14

ON PAGE 143

TWO-WAY LIST

Two-way list, which can be traversed in two directions; in the usual forward direction from the beginning of the list to the end, or in the backward direction from the end of the list to the beginning.

TWO-WAY LIST...(cont)

A two-way list is a linear collection of data elements, called nodes, where each node is divided into three parts:

1. An information field **INFO** which contains the data of N.
2. A pointer field **FORW** which contains the location of the next node in the list.
3. A pointer field **BACK** which contains the location of the preceding node in the list.